



Réveil Jour 2 — 15 min

PRATIQUE

1 Sans notes : recréer un mini-DAG `wakeup` (2 tâches Bash chaînées, `catchup=False`) et le lancer.

+ **Bonus** : 3^e tâche en parallèle.

JOUR 2

Data-aware, partage de données, tests & concepts avancés



AU PROGRAMME

- Sensors & Assets (data-aware)
- Event-driven scheduling
- XCom & templating Jinja
- Organisation du code & erreurs
- Tests et CI/CD
- TaskGroups, branching, mapping dynamique
- Variables & multi-environnements

04



Scheduling avancé & data-aware



Attendre un événement : les Sensors

01

Sensor = operator qui attend une condition (fichier, table...).
`FileSensor`.

02

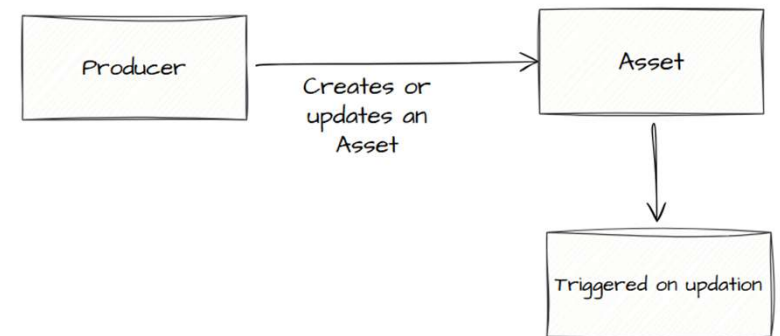
Modes : `poke` (occupe un slot) / `reschedule` (libère) / **deferrable** (0 slot, via Triggerer).

03

L'approche « classique » avant les Assets.

Data-Aware Scheduling : la notion d'Asset

- **Nouveauté majeure v3.** Un **Asset** = une donnée (table, fichier).
- Planifier par la *donnée* plutôt que par l'*heure* : un DAG se déclenche quand une donnée est mise à jour.





Producteur / consommateur (outlets, schedule)

01

Producteur :
`@task(outlets=[asset])`
ou `outlets=` sur l'operator.

02

Consommateur :
`schedule=[asset].`

03

Chaînage event-driven
sans sensor.



Dépendances orientées données (conditions ET/OU)

01

Chaîner tâches → Assets
→ DAGs.

02

Combiner plusieurs Assets
: conditions logiques **ET /**
OU, aliases, watchers.

03

Graphes de dépendances
data.



Exemple : disponibilité d'un fichier / d'une table

01

Fichier déposé → Asset
"fichier" → DAG
d'ingestion.

02

Table rafraîchie → Asset
"table" → DAG de
reporting.

03

Sensor vs Asset selon la
source.



ENCART 2.x → 3.x : Datasets → Assets

AIRFLOW 2 → 3

- Les **Datasets** d'Airflow 2.4+ deviennent les **Assets** en v3.
- API enrichie (conditions, aliases, watchers, event-driven). Concept élargi et central.

Assets : l'ordre d'activation compte

PIÈGE

SYMPTÔME

Le producteur passe au **vert**, mais le consommateur **ne démarre jamais**. L'interface affiche « **0 sur N assets mis à jour** ». Aucune erreur.

CAUSE

Deux causes se cumulent : un DAG **en pause** n'est pas déclenché, *même par un Asset* ; et Airflow ne compte que les événements survenus **après** l'enregistrement du consommateur.

SOLUTION

Activer les consommateurs D'ABORD, puis lancer le producteur. Pour la condition **ET**, lancer **les deux** producteurs.

*Bon réflexe de diagnostic : ouvrez la vue **Assets** de l'interface. Vous y verrez l'asset, ses événements horodatés et les DAGs qui le consomment — c'est là qu'on voit si l'événement a été émis, ou simplement ignoré.*

TP3 : Assets producteur / consommateur

PRATIQUE

25 MIN

- + Point de départ : votre dags/snowystore_etl.py du TP1 — tout se fait dans CE FICHIER
- 1 Mission 1 : y déclarer ventes = Asset("snowystore://sales"), et l'ajouter sur la tâche load : @task(outlets=[ventes])
- 2 Mission 2 : dans LE MÊME FICHIER, ajouter un 2^e DAG snowystore_reporting avec schedule=[ventes] au lieu d'un horaire
- 3 Mission 3 : ACTIVER D'ABORD le consommateur, PUIS lancer l'ETL — le reporting doit démarrer tout seul. Bonus : 2^e Asset clients et condition ET (ventes & clients)



TP3 : Restitution

CHECKPOINT

- ✓ Asset déclaré
- ✓ reporting déclenché par l'Asset (pas l'heure)
- ✓ (bonus) condition ET fonctionnelle.

Corrigé — TP3 : Assets producteur / consommateur

CORRIGÉ

```
from airflow.sdk import Asset, dag, task

snowystore_sales = Asset("snowystore://sales")

# --- PRODUCTEUR : la tache declare qu'elle met a jour la donnee ---
@task(outlets=[snowystore_sales])
def load_sales():
    ...

# --- CONSOMMATEUR : declenche par la DONNEE, pas par une heure ---
@dag(schedule=[snowystore_sales], catchup=False, start_date=...)
def tp3_consumer_reporting():
    ...

# Condition ET (bonus) : schedule=(asset_a & asset_b)
```

TP3 bis : Sensor et mode deferrable

PRATIQUE

20 MIN

- + Nouveau fichier : dags/tp3bis_sensor.py
- 1 Mission 1 : un FileSensor, filepath="../../../include/data/orders_{{ ds }}.csv", mode="reschedule", poke_interval=15
- 2 Mission 2 : le faire suivre d'une tâche qui logge. Lancer SANS le fichier : observer l'état up_for_reschedule
- 3 Mission 3 : créer le fichier, observer le déblocage. Comparer avec l'Asset du TP3 : lequel choisir, et pourquoi ?

Corrigé — TP3 bis : sensor et modes d'attente

CORRIGÉ

```
wait_for_file = FileSensor(  
    task_id="wait_for_orders_file",  
    filepath="/usr/local/airflow/include/data/orders_{{ ds }}.csv",  
    mode="reschedule",      # libere le slot entre deux verifications  
    poke_interval=15,  
    timeout=60 * 10,  
)  
  
# mode="poke" (defaut) : occupe un slot pendant TOUTE l'attente  
# mode="reschedule"    : libere le slot -> a privilegier si l'attente est longue  
# version deferrable    : 0 slot, l'attente est portee par le Triggerer  
  
wait_for_file >> process()
```



Event-Driven Scheduling (Airflow 3) : message queues

- **Nouveauté v3.** Déclencher un DAG depuis un **événement externe** via une message queue (ex : SQS, Kafka...).
- Le DAG réagit à un message, pas à une horloge ni à un Asset interne.



Autres modes : trigger via API & timedelta

01

Trigger via API REST
(déclenchement externe
programmatique, vu J1).

02

Scheduling par `timedelta`
(intervalle glissant).

03

Panorama complet des
modes de déclenchement.



Micro-exercice : choisir le mode de déclenchement

PRATIQUE

10 MIN

- 1 Pour 4 scénarios (rapport quotidien à heure fixe · reporting quand la table ventes change · ingestion quand un message arrive sur une file · lancement depuis une appli maison), choisir : **cron / Asset / event-driven / trigger API** et justifier
- 2 Implémenter le plus simple des quatre sur un DAG jetable

05



Partage de données & templating



XCom : principe & stockage

01

XCom = *Cross-Communication*.

02

Petit message entre tâches via la Metadata DB.

03

Pour de **petites** valeurs (ids, chemins, comptes), **pas** des DataFrames.

Push / pull & TaskFlow automatique

- `ti.xcom_push` / `ti.xcom_pull` (classique) ; le `return` d'une fonction est poussé automatiquement.
- En TaskFlow, passer un retour crée XCom **et** dépendance.





Limites & bonnes pratiques XCom

01

Stocké en DB → **limité en taille.**

02

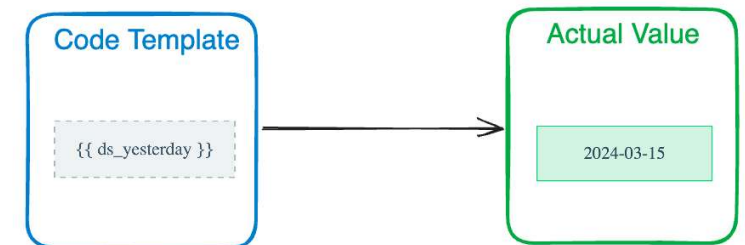
Vraie donnée : écrire sur S3/GCS, ne passer que le **chemin.**

03

Custom XCom backend pour automatiser. Jamais de secret en XCom.

Jinja templating : variables de contexte

- Champs *templated* rendus à l'exécution : `{{ ds }}` (date du run), `{{ data_interval_start }}` / end, `{{ params.x }}`. Chemins/dates dynamiques.



Macro gets replaced with actual value at runtime



Référentiel des templates & champs templated

01

Tous les champs d'un operator ne sont pas templated : voir `template_fields`.

02

Variables dispo : `ds`, `ts`, `data_interval_*`, `run_id`, `params`, `var`, `conn`, macros.

03

Doc « templates reference ».

TP4 : XCom + templating

PRATIQUE

30 MIN

- + Point de départ : votre dags/snowystore_etl_tasks.py (celui du tp1 en task flow)
- 1 Mission 1 : extract lit include/data/orders_<ds>.csv, compte les lignes et les RENVOIE (return = XCom automatique)
- 2 Mission 2 : transform et load reçoivent ce nombre en argument et l'affichent
- 3 Mission 3 : rendre le chemin dynamique avec {{ ds }} dans un champ templaté. Bonus : une Variable utilisée via {{ var.value.ma_cle }}



TP4 : Restitution

CHECKPOINT

10 MIN

- ✓ nombre de lignes propagé
- ✓ chemin dynamique par `{{ ds }}`
- ✓ borne de période affichée.

Corrigé — TP4 : XCom + templating Jinja

CORRIGÉ

```
@task
def extract(**context) -> int:
    ds = context["ds"]                # date de l'intervalle traite
    path = DATA_DIR / f"orders_{ds}.csv" # chemin dynamique
    with path.open(encoding="utf-8") as fh:
        n_rows = sum(1 for _ in csv.DictReader(fh))
    return n_rows                     # -> XCom automatique

# Dans un champ TEMPLATE (bash_command, sql, filepath...) :
BashOperator(
    task_id="show_date",
    bash_command="echo 'orders_{{ ds }}.csv | {{ data_interval_start }}'",
)
```



Questions & reprise — fin de matinée J2

01

Vos questions sur : Assets,
event-driven, XCom,
templating Jinja

02

Reprise des points signalés
comme flous

03

Partage d'écran : je
reprends en direct un point
à la demande

06



Organisation du code & gestion des erreurs



Organisation du dossier dags & perf du DAG Processor

- Fichiers relus en boucle → top-level léger. 1 DAG = 1 responsabilité.
- Éviter les fichiers monstres ; imports lourds hors du top-level.

Factorisation : la logique dans des modules

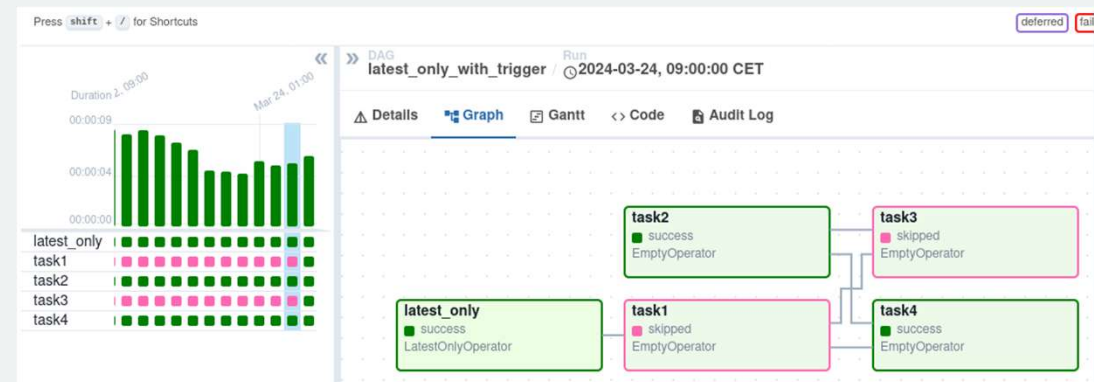
- Logique métier dans `include/` (ou packages importés), pas dans le fichier DAG.
- Le DAG *orchestre*, les modules *font*. Réutilisable & testable.



FACTORING

Gestion des erreurs : DAG-level & Task-level

- Task-level : `retries`, `retry_delay`, `backoff` exponentiel.
- DAG-level : `default_args`, `dagrun_timeout`.
retries n'ont de sens que si la tâche est **idempotente**.



Notifications, callbacks & self-healing

- Callbacks `on_success_callback` / `on_failure_callback` / `on_retry_callback` → Slack/email/PagerDuty.
- **Self-healing** : retries, clear auto, tâches de remédiation. Alerter sur l'actionnable.

New Callback 🤖

```
from airflow.exceptions import AirflowSkipException

def task_skipped(context):
    print(f'Task has been skipped, task_instance_key_str: {context['task_instance_key_str']}')

@dag(
    ...
)
def my_dag():

    @task(on_skipped_callback=task_skipped)
    def this_task_will_be_skipped():
        raise AirflowSkipException

    this_task_will_be_skipped()

my_dag()
```

TP5 : Factorisation + handlers d'erreur

PRATIQUE

30 MIN

- + Point de départ : votre dags/snowystore_etl.py (celui en taskflow)
- 1 Mission 1 : déplacer la logique de transform dans include/transform_logic.py, et l'IMPORTER depuis le DAG
- 2 Mission 2 : ajouter retries=2, retry_delay et retry_exponential_backoff dans les default_args
- 3 Mission 3 : écrire alerte(context), la brancher en on_failure_callback, et casser une tâche exprès pour la voir partir



TP5 : Restitution

CHECKPOINT

- ✓ logique factorisée & importée
- ✓ retries actifs (vus dans l'UI)
- ✓ alerte déclenchée sur échec simulé.

Corrigé — TP5 : factorisation & gestion d'erreurs

CORRIGÉ

```
from include.transform_logic import aggregate_by_category, clean_orders

def alert_on_failure(context):          # on_failure_callback
    ti = context["task_instance"]
    print(f"ALERTE | dag={ti.dag_id} | task={ti.task_id} | try={ti.try_number}")

@dag(default_args={
    "retries": 2,
    "retry_delay": timedelta(seconds=30),
    "retry_exponential_backoff": True,    # 30s, 1min, 2min...
    "on_failure_callback": alert_on_failure,
})
def snowstore_factorise():
    ...    # le DAG orchestre, include/ fait le travail
```

07



Tests & CI/CD



Pourquoi tester les DAGs et les tasks ?

01

Un DAG est du code → il casse comme du code.

02

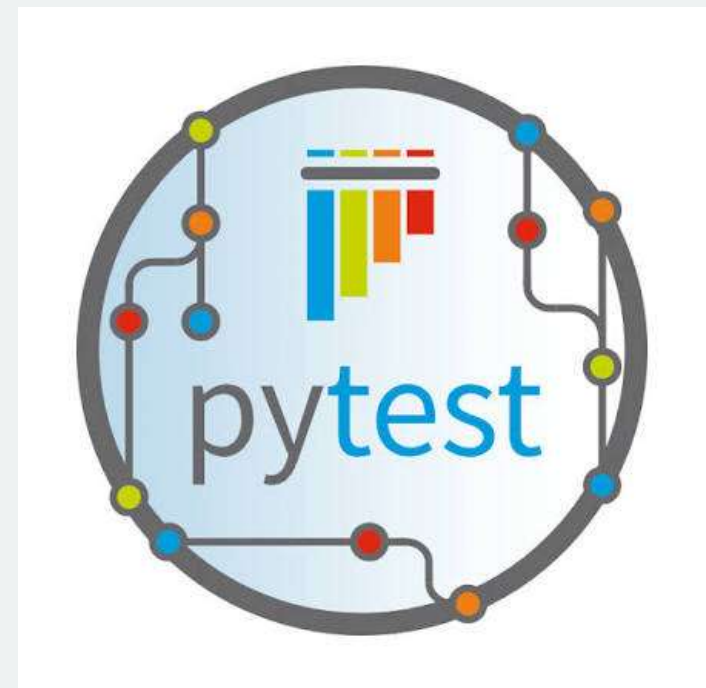
Un test d'import attrape
~80 % des bêtises.

03

Confiance pour livrer, non-
régression, migration
sereine.

Outils & commandes

- `pytest` pour les tests. `dag.test()` (run local complet sans scheduler). `airflow tasks test / airflow dags test` (CLI). Fixtures & mocking.



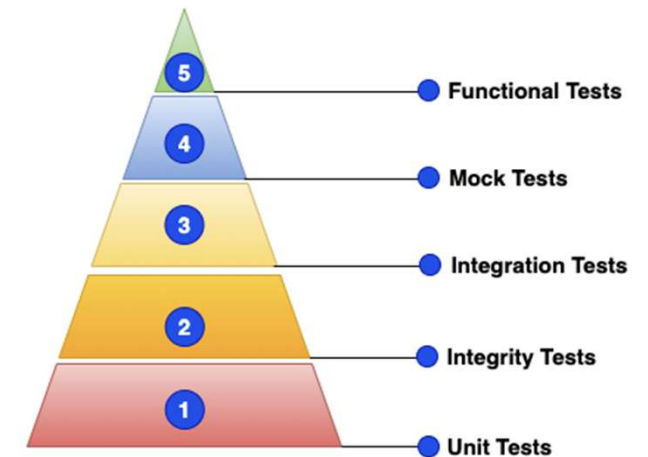


Sanity checks : validité des DAGs

- Test d'intégrité : tous les DAGs s'importent, pas de cycle, `dag_id` uniques, tags présents.
- À lancer en CI sur chaque commit.

Tests unitaires de tasks

- Tester la **logique métier** isolée (fonctions dans `include/`) : entrées connues → sortie attendue.
- Mock des dépendances externes. Rapide, ciblé.





CI/CD

- DAGs en Git
- revue de code
- tests en CI (import + unitaires)
- déploiement contrôlé (image / DAG bundle)
- dev → staging → prod.



Exercice : votre pipeline de CI

PRATIQUE

10 MIN

-  Point de départ : le fichier `.github/workflows/ci.yml` fourni dans le projet
-  Mission 1 : l'ouvrir et identifier ses 3 étapes — installation, test d'intégrité, tests unitaires
-  Mission 2 : y ajouter l'exécution de vos propres tests
-  Mission 3 : simuler la CI en local — dans le conteneur, lancer `pytest tests/ -v`

TP6 : Tester le pipeline

PRATIQUE

30 MIN

- 1 Mission 1 : écrire `tests/test_dag_integrity.py` — charger tous les DAGs avec DagBag, vérifier qu'aucun ne lève d'erreur
- 2 Mission 2 : écrire `tests/test_transform_logic.py` — tester la fonction extraite au TP5, SANS démarrer Airflow
- 3 Mission 3 : lancer `pytest tests/ -v` DANS le conteneur (astro dev bash au préalable)
- + Défi bonus : mocker l'accès à Postgres pour tester la logique sans vraie base



TP6 : Restitution

CHECKPOINT

- ✓ test d'import vert
- ✓ test unitaire vert
- ✓ `dag.test()` OK.

Corrigé — TP6 : les deux tests essentiels

CORRIGÉ

```
# tests/test_dag_integrity.py -- LE test le plus rentable
def test_no_import_errors():
    dagbag = DagBag(dag_folder="dags/", include_examples=False)
    assert not dagbag.import_errors, f"Erreurs : {dagbag.import_errors}"

# tests/test_transform_logic.py -- possible GRACE a la factorisation du TP5
def test_rejette_les_montants_negatifs():
    rows = [{"category": "ski", "amount": "-5"}]
    assert clean_orders(rows) == [] # sans demarrer Airflow, en < 1 s

# Lancer : astro dev bash puis pytest tests/ -v
```

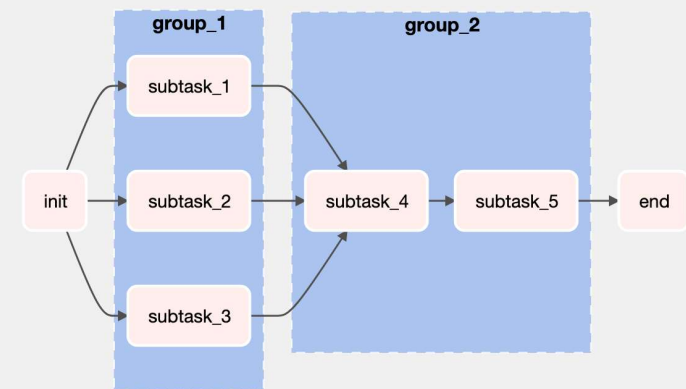
08



Concepts avancés de DAGs

Task Groups

- `with TaskGroup("nom")` : regrouper visuellement/logiquement, repliable dans l'UI, sans overhead. Remplace le SubDag.





Branching / branchement conditionnel

- `@task.branch / BranchPythonOperator` :
retourne le nom de la/des tâche(s) à suivre ;
les autres sont *skipped*.



Re-branchement du flux

- Quand les branches se rejoignent : la tâche de jointure hérite d'`all_success` → risque de *skip en cascade*.
- Solution :
`trigger_rule="none_failed_min_one_succes`
`s"`.



Trigger Rules avancées

- `all_success` (défaut)
- `all_done`
- `one_failed`
- `one_success`
- `none_failed_min_one_success`
- `all_skipped`. Cas : notification, nettoyage, jointure post-branch.

Dynamic Task Mapping

- `.expand()` : générer N tâches à l'exécution depuis une liste (1 par fichier/catégorie).
`.partial()` fige les arguments constants.
Fan-out dynamique.



TP7 : Branching, TaskGroups & mapping

PRATIQUE

30 MIN

- + Point de départ : votre dags/snowystore_etl.py (celui en taskflow)
- 1 Mission 1 : une tâche `@task.branch` qui renvoie "full_load" le lundi, "incremental_load" sinon. Créer les 2 cibles
- 2 Mission 2 : regrouper les transformations dans un TaskGroup transformations (attention : les task_id deviennent préfixés)
- 3 Mission 3 : une transformation par catégorie avec `.expand()`, la liste venant d'une tâche amont
- 4 Mission 4 : la jointure se fait skipper — corriger avec `trigger_rule="none_failed_min_one_success"`



TP7 : Restitution

CHECKPOINT

- ✓ branche correcte
- ✓ TaskGroup repliable
- ✓ N tâches mappées
- ✓ jointure exécutée (pas sautée).

Corrigé — TP7 : branching, TaskGroup & mapping

CORRIGÉ

```
@task.branch
def choose_load_mode(**context) -> str:
    weekday = context["data_interval_start"].weekday()
    return "full_load" if weekday == 0 else "incremental_load"

@task_group(group_id="transformations")
def transformations(categories):
    # .partial() fige les constantes, .expand() itere -> N taches a l'execution
    return transform_category.partial(run_env="formation").expand(category=categories)

# LE PIEGE : sans cette trigger_rule, join est SKIPPEE en cascade
# (les branches non choisies sont "skipped", pas "failed")
join = EmptyOperator(task_id="join",
                     trigger_rule="none_failed_min_one_success")
```



Exercice : la matrice des trigger rules

PRATIQUE

10 MIN

- 1 Sur un DAG jetable : 2 parents (un succès, un échec forcé) → tester `all_success`, `all_done`, `one_failed`, `one_success`
- 2 Noter pour chaque règle si l'enfant part ou non, et l'état obtenu
- 3 Conclure : quelle règle pour une **notification**, pour un **nettoyage**, pour une **alerte** ?

09



Variables, configuration & environnements



Variables Airflow : définition & usage

01

Couples clé/valeur globaux.

02

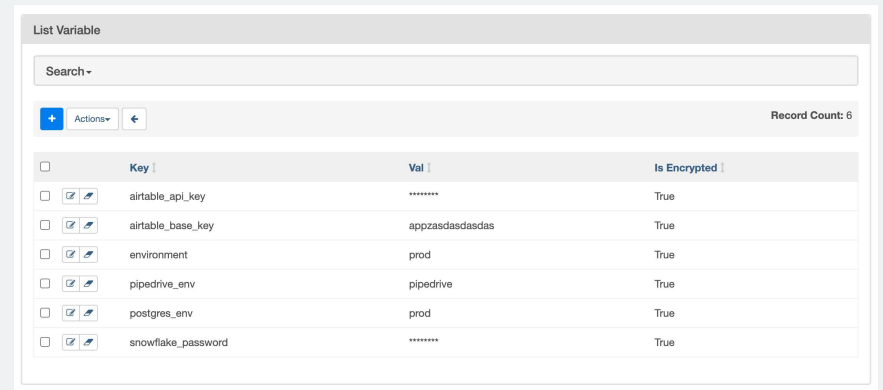
Définition : **UI**, **CLI**
(`airflow variables set`),
import JSON.

03

Usage : en code
(`Variable.get`) ou
templaté (`{{ var.value.x }}`). pas de secrets.

Gestion multi-environnements (dev / staging / prod)

- Un même code, des configs différentes par environnement (chemins, bases, volumes).
- Via Variables importées d'un **fichier de config par env**, ou variables d'environnement. Zéro **if prod**: en dur.



The screenshot shows a web interface titled "List Variable". It features a search bar at the top, a table with 6 records, and a "Record Count: 6" indicator. The table has columns for "Key", "Val", and "Is Encrypted". Each row includes a checkbox and edit icons. The variables listed are: airtable_api_key, airtable_base_key, environment, pipedrive_env, postgres_env, and snowflake_password.

	Key	Val	Is Encrypted
☐	airtable_api_key	*****	True
☐	airtable_base_key	appzasdasdasdas	True
☐	environment	prod	True
☐	pipedrive_env	pipedrive	True
☐	postgres_env	prod	True
☐	snowflake_password	*****	True



DAGs producer / consumer avec variables de config

- Un DAG *producteur* écrit une donnée dont le chemin/paramètres viennent d'une Variable de config ; un DAG *consommateur* la lit selon la même config.
- Découplage par la configuration.

TP8 : Producer / consumer + variables de config

PRATIQUE

20 MIN

- 1 Mission 1 : créer une Variable snowstore_config (Admin → Variables), en JSON, avec une clé output_dir
- 2 Mission 2 : nouveau fichier dags/tp8_producer_consumer.py — un DAG producteur qui ÉCRIT à l'emplacement lu dans la Variable
- 3 Mission 3 : dans LE MÊME FICHIER, un DAG consommateur qui LIT au même emplacement, via la même Variable
- 4 Mission 4 : changer la Variable dans l'UI, relancer : les 2 DAGs suivent SANS modification de code



TP8 : Restitution

CHECKPOINT

- ✓ config externalisée
- ✓ producer/consumer cohérents
- ✓ changement de config sans toucher au code.

Corrigé — TP8 : variables de configuration

CORRIGÉ

```
from airflow.sdk import Variable

def get_config() -> dict:
    # Airflow 3 : le parametre est "default" (et non "default_var")
    raw = Variable.get("snowystore_config", default=json.dumps(DEFAULT_CONFIG))
    return json.loads(raw)

# PRODUCTEUR et CONSOMMATEUR lisent la MEME config :
config = get_config()
target = Path(config["output_dir"]) / f"sales_{context['ds']}.json"

# Changer la Variable suffit -> les deux DAGs suivent, sans toucher au code.
# En template : {{ var.value.snowystore_env }}
```



Récap Jour 2 / Quiz flash

- 5 questions : Asset vs XCom ? event-driven vs data-aware ? pourquoi factoriser la logique hors du DAG ? le test le plus rentable ? piège du re-branchement ?



Questions & reprise — fin de journée J2

01

Vos questions sur :
organisation du code,
erreurs, tests, concepts
avancés, variables

02

Reprise des points signalés
comme flous

03

Partage d'écran : je
reprends en direct un point
à la demande